

I am learning K-Means I want you to write simplest python script so that I can implement and understand K-means. Use some online dataset that is good for K-means implementation.

Write the simplest code possible

Print graph

make sure to calculate the accuracy and loss

Python

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
from sklearn.metrics import adjusted_rand_score

# Load the Iris dataset (a classic dataset good for K-Means, available on
iris = load_iris()
X = iris.data # Features (sepal length, sepal width, petal length, petal
y_true = iris.target # True labels (for evaluation, though K-Means is un

# Initialize and fit K-Means with 3 clusters (since Iris has 3 species)
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans.fit(X)

# Get predicted cluster labels
y_pred = kmeans.labels_

# Calculate loss (inertia: sum of squared distances to centroids)
loss = kmeans.inertia_
print(f"Loss (Inertia): {loss}")

# Calculate accuracy using Adjusted Rand Index (compares clustering to tr
accuracy = adjusted_rand_score(y_true, y_pred)
print(f"Accuracy (Adjusted Rand Index): {accuracy}")

# Plot the clusters (using first two features for 2D visualization)
plt.figure(figsize=(8, 6))
plt.scatter(X[:, 0], X[:, 1], c=y_pred, cmap='viridis')
plt.title('K-Means Clustering on Iris Dataset')
```

```
plt.xlabel('Sepal Length')
plt.ylabel('Sepal Width')
plt.colorbar(label='Cluster')
plt.show()
```

Python

```
Y_Pred = kmeans.labels_
print(Y_Pred)
```

What is the meaning of this code

Answer in short

Short answer:

Python

```
Y_Pred = kmeans.labels_
print(Y_Pred)
```

Meaning:

`kmeans.labels_` gives you the **cluster number** (0, 1, 2, ...) that each data point was assigned to after running K-Means.

`Y_Pred` stores these cluster assignments (predicted "labels").

`print(Y_Pred)` shows which cluster each point belongs to.

Example output might look like:

```
[0 0 0 0 1 1 2 2 2 1 ...]
```

→ first 4 points → cluster 0, next 2 → cluster 1, etc.

Python

```
loss = kmeans.inertia_
print(loss)
```

Short explanation:

Python

```
loss = kmeans.inertia_
print(loss)
```

What it means:

- `kmeans.inertia_` is the **most important internal score** of K-Means
- It shows the **total sum of squared distances** between each point and its assigned cluster center (centroid)
- Also called **Within-Cluster Sum of Squares (WCSS)**

Smaller inertia = better clustering (points are closer to their centroids)

In practice:

Python

```
loss = kmeans.inertia_      # usually a big number like 78.9 or 152.3
print(loss)                 # people often call this value "the loss" of k
```

So when you see people talking about "K-Means loss", they almost always mean `inertia_`!

How is WCSS is calculate and why?

WCSS (Within-Cluster Sum of Squares) is the most important "loss" value in K-Means.

How is WCSS calculated? (very simple version)

For each cluster:

1. Take every point that belongs to this cluster
2. Calculate distance between the point and the cluster center (centroid)
3. Square that distance
4. Sum all these squared distances → this is the WCSS of **one cluster**

Finally:

WCSS total = sum of WCSS of all clusters

Mathematically:

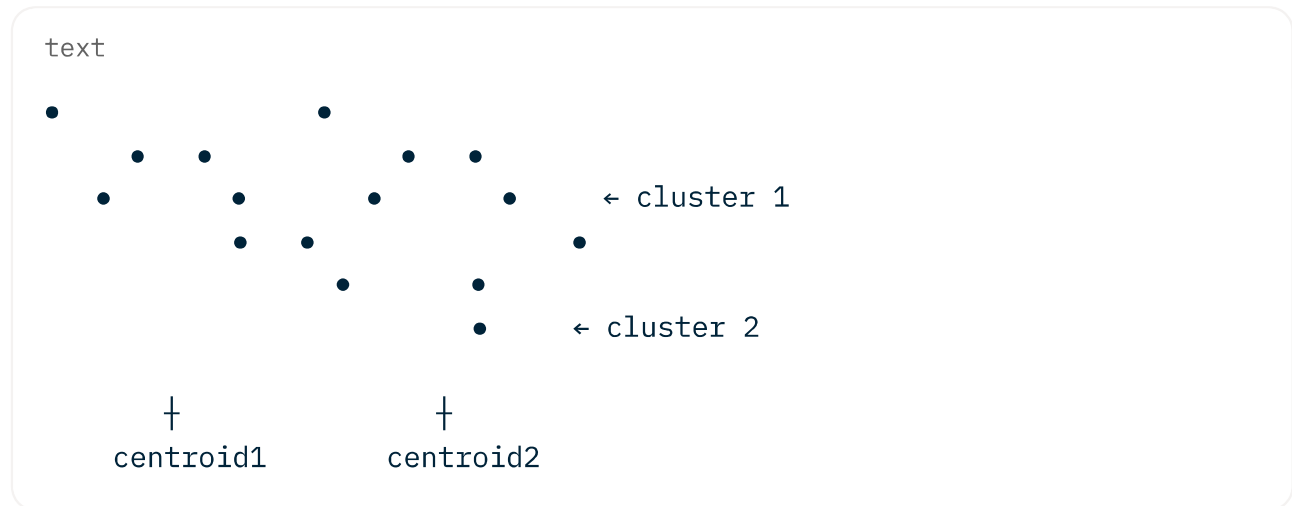
text

$$\text{WCSS} = \sum_{k=1 \text{ to } K} \sum_{\text{point in cluster } k} || \text{point} - \text{centroid}_k ||^2$$

($\|x\|^2$ means squared Euclidean distance)

Very visual explanation

Imagine this in 2D:



WCSS measures:

"How much do all the points **dislike** sitting far from their own centroid?"

(every extra distance hurts **quadratically** — that's why we square it)

Why do we square the distances? (important reasons)

Reason	Explanation
1. No negative distances	Distance can't be negative → squaring keeps everything positive
2. Punishes big mistakes more	Being 10 units away hurts 100× more than being 1 unit away ($10^2 = 100$)
3. Mathematically nice properties	Makes the function smooth → easier to optimize (gradient works well)
4. Same as variance	WCSS is basically total variance inside all clusters
5. Tradition	Almost all clustering papers/measures use squared Euclidean distance

Quick summary table

Goal of K-Means	What WCSS represents	Why we like small WCSS
Put similar points together	How spread out are points inside clusters	Smaller = points are tight together
Make clusters compact	Total "internal mess" of all clusters	The less mess → better clusters

Bottom line:

WCSS = total squared error inside all clusters

Smaller WCSS → better, more compact clusters

(That's why in the elbow method we want to see where WCSS stops decreasing fast)

↳ Elbow method for K

↳ Other clustering metrics